Univerzitet u Nišu
Elektronski fakultet u Nišu

Master akademske studije
Računarstvo i informatika
Modul: Softversko inženjerstvo

Seminarski rad iz predmeta
Sistemi za upravljanje bazama podataka

Obrada transakcija, planovi izvršavanja transakcija, izolacija i zaključavanje u Firebird bazi podataka

Profesor:
dr Aleksandar S. Stanimirović

Student:
Jefimija Stamenović 1820/24

Mart, 2026. godine

Sadržaj

1. Uvod 4
   1.1. Koncept transakcije kao osnovnog stuba pouzdanosti informacionih sistema 4
   1.2. Izazovi paralelizma: Sukobi pri istovremenom pristupu podacima u višekorisničkom okruženju 4
   1.3. Važnost mehanizama izolacije za stabilnost poslovnih aplikacija 5
   1.4. Predmet i cilj rada: Analiza specifičnosti Firebird transakcionog modela i MGA arhitekture 5
2. ACID principi i nivoi izolacije 6
   2.1. Detaljna dekompozicija ACID svojstava 6
   2.2. Transakcione anomalije 7
      2.2.1. Prljavo čitanje 7
      2.2.2. Neponovljivo čitanje 7
      2.2.3. Fantomsko čitanje 7
   2.3. Nivoi izolacija transakcija 8
      2.3.1. Read Uncommitted 8
      2.3.2. Read Committed 8
      2.3.3. Repeatable Read 9
      2.3.4. Serializable 9
   2.4. Odnos nivoa izolacije i transakcionih anomalija 10
3. Multi-generational Architecture (MGA) 11
   3.1. Multi-Version Concurrency Control (MVCC) 11
   3.2. Mehanizam verzija zapisa: Kako se čuva istorija izmena bez blokiranja čitalaca? 11
   3.3. Copy-on-Write princip: Fizički smeštaj novih verzija podataka na stranice baze 12
   3.4. Transaction Inventory Page (TIP) 13
   3.5. Record Header: Meta-podaci o Transaction ID-u na nivou svakog reda 14
4. Implementacija nivoa izolacije u Firebird-u 15
   4.1. Snapshot 15
   4.2. Read Committed 15
      4.2.1. Razlika između RECORD VERSION i NO RECORD VERSION 15
   4.3. Snapshot Table Stability 16
   4.4. Read Committed Read Consistency 16

5. Mehanizmi zaključavanja (locking) i rešavanje konflikata 17
5.1. Pesimistička kontrola 17
5.2. Optimistički pristup 17
5.3. Arhitektura i uloga Lock Manager-a 18
5.4. Deadlock u Firebirdu: Algoritmi detekcije i strategije izbegavanja 18
5.5. Parametri čekanja: Wait, No Wait i uticaj Lock Timeout parametra 18
6. Održavanje baze i garbage collection 19
6.1. Problem rasta baze podataka (Database Bloat) usled akumulacije verzija zapisa 19
6.2. Analiza transakcionih markera 20
6.2.1. Definicija markera i njihova uloga 20
6.2.2. SQL monitoring markera u realnom vremenu 21
7. Planovi izvršavanja (Execution Plans) i monitoring 24
7.1. Uloga TPB (Transaction Parameter Block) u donošenju odluka 24
7.2. Uticaj nivoa izolacije na izbor putanje: Indeksi vs. Natural Scan 24
7.3. Dijagnostika transakcija u realnom vremenu putem sistemskih tabela (MON$TRANSACTIONS) 25
7.4. Zašto je Snapshot sporiji kod "dugih" transakcija? 26
8. Praktična demonstracija i studija slučaja 26
9. Zaključak 33
10. Literatura 34

1. Uvod

1.1. Koncept transakcije kao osnovnog stuba pouzdanosti informacionih sistema

U svetu savremenog razvoja softvera gde se podaci generišu i menjaju u milisekundama, pitanje integriteta baze podataka prestalo je da bude samo tehnička karakteristika i postalo je osnova poslovne stabilnosti. Svaki put kada korisnik izvrši neku operaciju – bilo da je to prenos novca sa jednog računa na drugi, rezervacija avionske karte ili prosta izmena statusa narudžbenice u e-commerce sistemu – mi se oslanjamo na koncept transakcije.

Transakcija se najjednostavnije može definisati kao logička jedinica rada koja okuplja jednu ili više operacija nad podacima u neraskidivu celinu. Formalno, transakcija predstavlja logičku jedinicu rada koja obuhvata jedan ili više SQL upita koji se izvršavaju kao nedeljiva celina. To znači da, ili će se sve operacije unutar transakcije izvršiti uspešno ili neće nijedna.

Suština transakcionog rada nije samo u tome da se podaci upišu, već da se garantuje da će baza uvek ostati u stabilnom i predvidljivom stanju, bez obzira na to da li je operacija uspela, da li je došlo do greške u softveru ili je server iznenada ostao bez napajanja. Bez snažnog transakcionog mehanizma, baze podataka bi brzo postale skup kontradiktornih informacija, gde bi se, na primer, novac skinuo sa jednog računa, a nikada ne bi pojavio na drugom. Transakcija je taj "zaštitni omotač" koji osigurava da se ili sve planirane promene izvrše u potpunosti, ili da se, u slučaju bilo kakvog problema, baza vrati u stanje u kojem je bila pre početka procesa.

Transakcija prolazi kroz nekoliko faza – početak, izvršavanje čitanja i zapisa, a zatim se potvrđuje (COMMIT) ili poništava (ROLLBACK). Za robustan dizajn sistema je od preke važnosti eksplicitno rukovanje greškama i politika ponovnog pokušaja (retry) kod konflikata, kao i razumevanje nivoa izolacije koji određuju vidljivost promena između konkurenata

1.2. Izazovi paralelizma: Sukobi pri istovremenom pristupu podacima u višekorisničkom okruženju

Problem upravljanja transakcijama postaje eksponencijalno teži onog trenutka kada bazi pristupi više korisnika istovremeno. U idealnom, ali nerealnom svetu, transakcije bi se izvršavale jedna po jedna (serijski). Međutim, u realnim sistemima sa stotinama ili hiljadama paralelnih konekcija, performanse bi bile neprihvatljivo niske ako bismo naterali korisnike da čekaju jedan na drugog.

Zbog toga moderni sistemi, poput Firebird-a, moraju da podrže paralelizam (concurrency). Tu dolazimo do ključnog izazova: kako dozvoliti dvojici korisnika da menjaju isti podatak u isto vreme, a da pritom jedan drugom ne "pregaze" izmene ili, što je još češće, da jedan korisnik ne čita podatke koje je drugi tek počeo da menja, a možda će od njih odustati. Ovi sukobi nisu samo teoretska mogućnost; oni su svakodnevica u razvoju aplikacija. Ako sistem za upravljanje bazom podataka (RDBMS) ne upravlja ovim sukobima na pametan način, dolazi do situacija gde se gube ažurirani podaci (Lost Updates) ili se donose pogrešne poslovne odluke na osnovu informacija koje su trenutno u procesu promene.

1.3. Važnost mehanizama izolacije za stabilnost poslovnih aplikacija

Izolacija je deo transakcione logike koji određuje koliko jedna transakcija sme da "vidi" ono što druge transakcije trenutno rade. To je zapravo balansiranje između dve suprotstavljene potrebe: brzine rada i tačnosti podataka. Što je izolacija stroža, to su podaci sigurniji, ali je sistem sporiji jer transakcije moraju više da čekaju ili da troše više resursa na proveru verzija.

Za programere i arhitekte baza podataka, razumevanje izolacije je kritično jer direktno utiče na ponašanje aplikacije u kriznim momentima. Loše odabran nivo izolacije može dovesti do toga da aplikacija radi savršeno tokom testiranja sa jednim korisnikom, ali da počne da izbacuje nasumične greške ili prikazuje pogrešne totalne sume čim se broj korisnika poveća. Stabilna poslovna aplikacija ne sme da zavisi od sreće u redosledu pristupa podacima; ona mora biti projektovana tako da transakcioni mehanizam baze automatski rešava ove konflikte, ostavljajući programeru fokus na poslovnu logiku, a ne na ručno zaključavanje tabela.

1.4. Predmet i cilj rada: Analiza specifičnosti Firebird transakcionog modela i MGA arhitekture

Iako svi veći sistemi za baze podataka teže istim ciljevima, načini na koji ih postižu se drastično razlikuju. Firebird se u ovom domenu izdvaja svojom specifičnom arhitekturom pod nazivom **Multi-Generational Architecture (MGA)**. Za razliku od nekih drugih popularnih sistema koji koriste komplikovane logove (Transaction Logs) da bi "vratili film" unazad u slučaju greške, Firebird koristi princip verzija zapisa.

Cilj ovog rada je da detaljno istraži kako Firebird upravlja transakcijama, kako njegovi specifični planovi izvršavanja utiču na brzinu obrade i na koji način mehanizmi zaključavanja sprečavaju konflikte. Poseban fokus biće na razumevanju toga kako Firebird postiže da čitaoci podataka nikada ne blokiraju pisače i obrnuto – što je jedna od njegovih najznačajnijih prednosti. Kroz analizu unutrašnjih struktura kao što su TIP (Transaction Inventory Page) i različitih nivoa izolacije, ovaj rad će pokušati da pruži jasan uvid u to kako se teoretski principi transakcija pretvaraju u praktično, robusno i efikasno rešenje unutar Firebird ekosistema.

2. ACID principi i nivoi izolacije

Da bismo razumeli specifičnost Firebird baze podataka, neophodno je da prvo definišemo standard na kojima leži svaki relacioni sistem za upravljanje bazama podataka. Upravljanje transakcijama poštuje strogo definisan skup pravila koji garantuju da će podaci preživeti čak i najteže sistemske greške.

2.1. Detaljna dekompozicija ACID svojstava

Skraćenica ACID predstavlja četiri ključna elementa na kojima počiva pouzdanost svake moderne baze podataka. Svako od sledećih svojstava rešava specifičan problem koji se javlja u radu sa podacima:

1) **Atomičnost (engl. Atomicity) –** posmatra transakciju kao operaciju „sve ili ništa". Baza ne sme dozvoliti „polovične" rezultate koji bi narušili logičku celinu podataka. Konkretan primer bi bio prenos novca sa računa A na račun B. Transakcija se sastoji od dve operacije: skidanja iznosa sa računa A i dodavanja istog iznosa na račun B. Ako sistem otkaže nakon prve operacije, novac bi nestao iz sistema. Atomičnost garantuje da će se u tom slučaju prva operacija poništiti (*rollback*) i vratiti početno stanje.
2) **Konzistentnost (engl. Consistency) –** osigurava da transakcija prevodi bazu iz jednog validnog stanja u drugo validno stanje pri čemu ne sme doći do kršenja nekog pravila integriteta kao što je strani ključ ili tip podatka. Na primer, ako u bazi postoji pravilo da stanje na računu ne sme biti negativno, tj. manje od 0, transakcija koja pokušava da isplati više novca nego što ima na računu biće automatski prekinuta. Baza odbija da prihvati promenu koja bi narušila ovo pravilo čime čuva logičku ispravnost podataka.
3) **Izolacija (engl. Isolation)** – ovo je možda i najkompleksnije svojstvo u višekorisničkom radu. Ono garantuje da operacije koje se izvršavaju istovremeno neće uticati jedna na drugu na nepredviđen način. Svaka transakcija treba da se „oseća" kao da je jedina u sistemu. Stepen te "privatnosti" varira u zavisnosti od nivoa izolacije koji odaberemo, što je ključna tema ovog rada. Ako imamo jednu transakciju koja računa ukupnu sumu svih faktura u bazi (što traje nekoliko sekundi) i drugu transakciju koja u istom trenutku menja iznos na jednoj fakturi,onda će izolacija obezbediti da prva transakcija dobije tačan rezultat (ili stanje pre ili stanje posle izmene), a ne neku nasumičnu, polovičnu sumu.
4) **Trajnost (engl. Durability):** Kada sistem potvrdi da je transakcija uspešno završena (commit), te promene postaju trajne i nepromenljive. Čak i ako server padne sekundu nakon potvrde, podaci moraju biti bezbedno sačuvani na disku ili u logovima tako da se mogu rekonstruisati pri ponovnom pokretanju. Na primer, nakon što korisnik klikne na „Sačuvaj" i dobije potvrdu od baze, on može biti siguran da su podaci upisani na disk. Čak i ako bi se server fizički ugasio (nestanak struje) delić sekunde kasnije, podaci će biti tu kada se sistem ponovo pokrene, jer je baza već osigurala njihov trajni zapis.

2.2. Transakcione anomalije

Kada se u sistemu omogući paralelni rad više korisnika nad istim setom podataka, a pritom se ne koristi maksimalni nivo izolacije, neminovno dolazi do sukoba između performansi i tačnosti. Sekvencijalno izvršavanje transakcija bi bilo najsigurnije, ali bi u praksi dovelo do zagušenja baze. Zato se u realnim sistemima dopušta preplitanje transakcionih operacija, što sa sobom povlači rizik od pojave **transakcionih anomalija**.

Suština ovih anomalija je u tome što jedna transakcija može postati svesna promena koje druga transakcija vrši pre nego što su te promene konačno potvrđene. U poslovnim aplikacijama, ovakva narušavanja izolacije mogu dovesti do ozbiljnih grešaka u podacima, kao što su netačni finansijski bilansi ili prodaja robe koja je u tom trenutku već procesuirana u drugom nalogu.

2.2.1. Prljavo čitanje

Do **prljavog čitanja (engl. Dirty Read)** dolazi kada jedna transakcija čita podatke koje je druga transakcija izmenila, ali još uvek nije potvrdila tj. nije urađen *commit*. Problem nastaje ako druga transakcija na kraju odustane od izmena tj. uradi *rollback*. U tom slučaju, prva transakcija je pročitala "prljave" podatke, radi sa podacima koji su bili privremeni i koji formalno nikada nisu postali stalni deo baze. Ovo je najopasnija anomalija i većina modernih baza, uključujući Firebird, podrazumevano je ne dozvoljava.

Primer: Prodavac u salonu automobila menja cenu vozila sa 20.000 na 15.000 evra, ali još nije potvrdio unos. U tom trenutku, kupac putem aplikacije vidi sniženu cenu i pokreće proces kupovine. Ukoliko prodavac uoči grešku i poništi transakciju, kupac je praktično doneo odluku na osnovu cene koja je bila „prljava” i nepostojeća.

2.2.2. Neponovljivo čitanje

Neponovljivo čitanje (engl. Non-repeatable Read) je anomalija koja nastaje kada jedna transakcija u toku svog trajanja dva puta čita isti red podataka, ali dobija različite rezultate. Razlog je taj što je između dva čitanja neka druga transakcija izmenila taj red i uspešno potvrdila promenu. Iako su podaci koje transakcija čita validni, ona gubi svoju unutrašnju konzistentnost jer se stanje objekta promenilo usred procesa obrade.

Primer: Program za generisanje izveštaja očita dug klijenta od 1.000 dinara. Dok proces pripreme izveštaja traje, klijent na drugom mestu izvrši uplatu i ta promena se proknjiži. Ako program pre same finalizacije ponovo proveri isto polje, videće nulu. Rezultat je dokument koji sadrži kontradiktorne vrednosti za istu stavku.

2.2.3. Fantomsko čitanje

Fantomsko čitanje (engl. Phantom Read) je specifičan oblik gde se ne menja vrednost postojećeg reda, već se menja čitav skup podataka koji zadovoljava određeni uslov. To se dešava kada se u toku trajanja jedne transakcije pojave novi zapisi (Insert) koji odgovaraju istom kriterijumu pretrage.

Primer: Menadžer zatraži listu svih zaposlenih sa platom većom od 100.000 dinara i dobije spisak od 10 imena. Dok on pregleda tu listu, kadrovska služba unese novog radnika sa platom od 110.000

dinara i potvrdi unos. Kada menadžer u okviru iste transakcije pokrene komandu za izračunavanje ukupnog budžeta, dobiće sumu za 11 ljudi, iako mu je početna lista prikazala deset.

2.3. Nivoi izolacija transakcija

Nivoi izolacije predstavljaju mehanizam kojim baza podataka kontroliše vidljivost promena između transakcija koje se izvršavaju u isto vreme. Njihova glavna uloga je sprečavanje anomalija, ali uz svesno prihvatanje kompromisa—što je izolacija stroža, to je konzistentnost podataka veća, ali su performanse lošije jer sistem troši više resursa na zaključavanja i proveru verzija. Standardni model prepoznaje četiri nivoa: *Read Uncommitted*, *Read Committed*, *Repeatable Read* i *Serializable*.

Važno je istaći da Firebird, zahvaljujući svojoj specifičnoj arhitekturi verzija zapisa (**MGA – Multi-Generational Architecture**), ove nivoe ne rešava na klasičan način putem fizičkog zaključavanja redova. Umesto da čitaoci blokiraju pisce, Firebird svakoj transakciji isporučuje odgovarajuću verziju podatka. To omogućava bazi da postigne visoku izolaciju bez drastičnog pada brzine, o čemu će biti više reči u poglavlju o unutrašnjoj arhitekturi.

2.3.1. Read Uncommitted

Read Uncommitted predstavlja najslabiji nivo izolacije u relacionim bazama podataka, gde je fokus gotovo isključivo na performansama, dok se integritet podataka stavlja u drugi plan. Glavna karakteristika ovog nivoa je da transakcija može da pristupi izmenama koje su drugi korisnici izvršili, ali ih još uvek nisu zvanično potvrdili putem *commit* naredbe. Zbog ovakve popustljivosti, ovde su teorijski dozvoljene sve poznate transakcione anomalije, a naročito prljavo čitanje, što ovaj nivo čini neprikladnim za finansijske i druge kritične poslovne sisteme. Upravo zbog visokog rizika po konzistentnost, moderni DBMS sistemi, uključujući i Firebird, ovaj nivo često uopšte ne podržavaju u izvornom obliku ili ga automatski prebacuju na stroži nivo izolacije. Njegova primena je danas svedena na specifične slučajeve gde su podaci informativnog karaktera i gde povremena netačnost ne izaziva sistemski problem, kao što je prikupljanje telemetrijskih podataka ili očitavanje vrednosti sa senzora u realnom vremenu.

Primer: Transakcija A ažurira iznos na bankovnom računu sa 1000 na 2000 dinara, ali proces još uvek traje i izmena nije potvrđena. U tom trenutku se pokreće transakcija B, koja radi na nivou Read Uncommitted, i ona odmah očitava novi iznos od 2000 dinara. Međutim, ukoliko se u transakciji A dogodi greška i ona izvrši ROLLBACK, izmena se poništava i stanje na računu se vraća na 1000. Transakcija B je u međuvremenu već procesuirala podatak od 2000 dinara, što znači da je radila sa "fantomskom" vrednošću koja formalno nikada nije ni postojala kao trajni zapis u bazi podataka, čime se direktno manifestuje anomalija prljavog čitanja.

2.3.2. Read Committed

Read Committed je najrasprostranjeniji nivo izolacije u produkcionim okruženjima jer nudi optimalan balans između brzine rada i sigurnosti podataka. Osnovno pravilo ovog nivoa je da transakcija može videti isključivo one promene koje su druge transakcije uspešno potvrdile pre samog trenutka čitanja. Time se efikasno eliminiše pojava prljavog čitanja, jer korisnik nikada ne dolazi u kontakt sa privremenim ili neproverenim stanjima baze. Ipak, ovaj nivo ne garantuje stabilnost podataka tokom celog životnog ciklusa transakcije; on štiti od čitanja tuđih radnih

verzija, ali ne sprečava situaciju da se podaci promene u intervalu između dva upita unutar iste sesije. Zbog toga je pogodan za sisteme gde je važno da se vidi trenutno, realno stanje baze, uz svest o tome da se konzistentnost na nivou ponovljenih upita žrtvuje radi veće propusne moći sistema.

Primer: Transakcija A započinje izmenu cene artikla u bazi podataka, ali ne izvršava COMMIT. Paralelno sa njom, transakcija B pokreće upit za očitavanje cene istog artikla i dobija staru vrednost od 1000 dinara, čime je potvrđeno da su nepotvrđene izmene skrivene. Međutim, onog trenutka kada transakcija A potvrdi svoj rad (COMMIT), nova cena od 2000 dinara postaje trajna. Ukoliko transakcija B, koja je još uvek aktivna, ponovo pokrene identičan upit za istu stavku, ona će sada dobiti vrednost od 2000 dinara. Ovakav scenario pokazuje da unutar jedne transakcije dva identična čitanja mogu vratiti različite rezultate, što ukazuje na prisustvo anomalije neponovljivog čitanja, koja je na ovom nivou izolacije dozvoljena.

2.3.3. Repeatable Read

Repeatable Read je nivo izolacije koji garantuje da će ponovna čitanja istih redova unutar jedne transakcije vratiti identične vrednosti, bez obzira na to što druge transakcije u međuvremenu vrše i potvrđuju izmene nad tim istim podacima. Ovaj nivo eliminiše anomalije prljavog i neponovljivog čitanja, pružajući korisniku stabilan pogled na podatke tokom čitavog procesa obrade. U sistemima kao što je Firebird, ovaj nivo se implementira kroz **SNAPSHOT** režim rada, gde svaka transakcija operiše sa logičkom slikom baze koja je nastala u trenutku njenog pokretanja. Sve naknadne modifikacije drugih korisnika ostaju nevidljive za trenutnu transakciju, čime se postiže visoka konzistentnost, ali uz cenu povećane potrošnje memorije zbog čuvanja starih verzija zapisa.

Prednost ovog nivoa je u tome što omogućava dugotrajne operacije, poput generisanja kompleksnih izveštaja, bez rizika da će se podaci promeniti "u hodu" i dovesti do kontradiktornih rezultata. Ipak, standardna definicija ovog nivoa izolacije i dalje ostavlja mogućnost pojave fantomskog čitanja, gde se broj redova koji ispunjavaju određeni kriterijum može promeniti ako druga transakcija ubaci nove zapise. Zbog specifičnosti MGA arhitekture, Firebird ovaj problem rešava efikasnije od većine drugih DBMS-ova, ali se generalno smatra da je Repeatable Read korak ispod potpune serijalizacije jer se fokusira na očuvanje stanja već pročitanih redova.

Primer: Transakcija A izvršava upit `SELECT * FROM racuni WHERE stanje > 1000;` i kao rezultat dobija tri reda podataka. Dok transakcija A još uvek traje, transakcija B u međuvremenu ubacuje novi red sa iznosom 1500 i odmah potvrđuje promenu (COMMIT). Ukoliko transakcija A ponovo izvrši isti upit tokom svog trajanja, ona i dalje vidi isključivo prvobitna tri reda. Novi red koji je dodala transakcija B postaće vidljiv tek nakon što se transakcija A završi i pokrene se potpuno nova sesija, čime je osigurano da radni skup podataka unutar transakcije A ostane nepromenjen.

2.3.4. Serializable

Serializable predstavlja najstroži nivo izolacije koji obezbeđuje potpunu izolaciju tako što simulira sekvencijalno izvršavanje transakcija. U ovom režimu, baza garantuje da će krajnji rezultat paralelnog rada više korisnika biti identičan onome koji bi se dobio da su se te transakcije izvršavale jedna po jedna, a ne istovremeno. Ovaj nivo eliminiše apsolutno sve tipove anomalija, uključujući prljavo, neponovljivo i fantomsko čitanje. Za razliku od nižih nivoa, Serializable ne

samo da "zamrzava" pogled na postojeće podatke, već aktivno štiti integritet čitavog skupa podataka koji je transakcija obradila, sprečavajući bilo kakve međusobne uticaje koji bi narušili logičku doslednost sistema.

U praksi se ovaj nivo realizuje na dva osnovna načina:

- **Pesimistički pristup –** podrazumeva korišćenje strogih zaključavanja nad čitavim opsezima tabela ili indeksnim ključevima, čime se fizički sprečavaju drugi korisnici da vrše izmene.
- **Optimistički pristup** – pristup bliži Firebird filozofiji i nasuprot pesimističkom pristupu, on dozvoljava rad, ali vrši strogu proveru konflikata u trenutku potvrđivanja izmena.

Cena ovakve sigurnosti je značajno smanjen paralelizam i povećana latencija, jer sistem troši dodatne resurse na validaciju svakog koraka. Zbog toga se ovaj nivo koristi isključivo za najkritičnije operacije, kao što su složeni finansijski transferi gde je očuvanje apsolutnog integriteta podataka prioritet u odnosu na brzinu odziva.

Primer: Transakcija A započinje rad i čita sve račune sa iznosom većim od 1000 dinara, pri čemu pronalazi tri odgovarajuća reda. Dok je transakcija A još uvek aktivna, transakcija B ubacuje potpuno novi račun sa iznosom od 1500 dinara i odmah potvrđuje promenu (COMMIT). Transakcija A, zahvaljujući SERIALIZABLE izolaciji, i dalje NE VIDI taj novi red kada ponovi svoj SELECT – time je izbegnuta pojava "fantoma" (phantom read). Međutim, ako transakcija A pokuša da IZMENI ili UPIŠE podatke**, situacija zavisi od implementacije:

- U nekim bazama (npr. PostgreSQL, Oracle) – transakcija A će uspeti, jer SERIALIZABLE samo štiti čitanje, ne sprečava upise
- U Firebirdu, sa SNAPSHOT TABLE STABILITY (najstroži nivo), transakcija B ne bi ni mogla da ubaci novi red dok A traje – bila bi blokirana ili bi dobila grešku odmah

2.4. Odnos nivoa izolacije i transakcionih anomalija

Sada kada smo definisali standardne nivoe izolacije, možemo pogledati tablicu u kojoj vidimo koje određene anomalije ne mogu da se pojave na određenom nivou izolacije:

Anomalija	**READ COMMITTED**	**SNAPSHOT (REPEATABLE READ)**	**SERIALIZABLE**
Dirty read	Ne	Ne	Ne
Non-repeatable read	Da	Ne	Ne
Phantom read	Da	Ne	Ne
Lost update	Da	Da	Ne
Write skew	Da	Da	Ne

Tabela 1:Anomalije vs Standardni nivoi izolacije

3. Multi-generational Architecture (MGA)

Za razliku od tradicionalnih relacionih baza koje integritet podataka održavaju komplikovanim sistemima zaključavanja (locking), Firebird se oslanja na specifičan koncept poznat kao **Multi-Generational Architecture (MGA)**. Osnovna filozofija ovog pristupa je da baza podataka u svakom trenutku čuva više verzija (generacija) istog zapisa. Ovakva struktura direktno rešava problem konkurentnosti, omogućavajući sistemu da opsluži ogroman broj istovremenih korisnika bez međusobnog blokiranja procesa čitanja i upisa.

3.1. Multi-Version Concurrency Control (MVCC)

Teorijski temelj na kojem počiva MGA je **Multi-Version Concurrency Control (MVCC)**. Glavna ideja ovog modela je eliminacija čekanja i to na sledeći način – umesto da transakcija koja želi da izmeni podatak "zaključa" taj red i primora sve ostale korisnike na čekanje, sistem kreira novu verziju tog istog podatka.

U praktičnom smislu, ovo znači da čitaoci (transakcije koje samo pregledaju podatke) nikada ne čekaju pisce (transakcije koje menjaju podatke). Kada čitalac zatraži podatak koji je trenutno u procesu izmene od strane drugog korisnika, Firebird mu isporučuje poslednju potvrđenu (committed) verziju koja je postojala u trenutku kada je čitalac započeo svoju transakciju. Time se uklanja jedno od najvećih uskih grla u arhitekturi baza podataka – blokiranje resursa usled konflikta između procesa analize i procesa ažuriranja podataka.

3.2. Mehanizam verzija zapisa: Kako se čuva istorija izmena bez blokiranja čitalaca?

U Firebirdu, operacija **UPDATE** ne funkcioniše po principu prepisivanja podataka na istoj fizičkoj lokaciji (tzv. *in-place update*). Svaki put kada se izvrši izmena nad određenim podacima, Firebird kreira novu fizičku verziju zapisa, dok prethodna verzija ostaje netaknuta. Rezultat ovakvog pristupa je da svaki logički red u tabeli zapravo predstavlja vremensku seriju verzija (generacija), koje su međusobno povezane u lanac. Ovaj lanac funkcioniše po principu jednostruko povezane liste, gde najnovija verzija zapisa u svom zaglavlju nosi pokazivač (*Back Interest Pointer*) na svoju neposrednu prethodnu verziju.

Kada *engine* pristupi određenom redu, on započinje proces validacije verzije kako bi odredio koji podatak je relevantan za datog korisnika. Firebird prolazi kroz lanac verzija i traži onu verziju koja je bila u stanju "*committed*" u trenutku startovanja tekuće transakcije. Sve verzije koje su kreirale transakcije sa statusom "*active*" (one koje su još uvek u toku) ili "*rolled back*" (one koje su poništene), sistem jednostavno ignoriše. Detaljan algoritam za navigaciju kroz ovaj lanac bi izgledao ovako:

- **Pozicioniranje na vrh** – baza uvek prvo pristupa najnovijoj verziji zapisa koja se nalazi na stranici podataka.
- **Provera statusa** – engine očitava *Transaction ID* (TID) iz zaglavlja te verzije i proverava status te transakcije na TIP stranici.

- **Evaluacija vidljivosti** – na osnovu nivoa izolacije (npr. *Snapshot*), baza odlučuje da li je ta verzija "vidljiva". Ako je transakcija koja je kreirala tu verziju još uvek aktivna ili je poništena, ta verzija se odbacuje.
- **Navigacija unazad** – ukoliko najnovija verzija nije validna za trenutnog korisnika, baza prati *Back Interest Pointer* i prelazi na prethodnu verziju u lancu. Ovaj proces se ponavlja sve dok se ne pronađe generacija zapisa koja je bila potvrđena u trenutku kada je korisnikova transakcija započela svoj rad.

Ovakav mehanizam omogućava da različite transakcije istovremeno vide različite verzije istog reda, što je ključno za postizanje izolacije bez upotrebe katanaca (*locks*) na samim podacima. Čitaoci nikada ne blokiraju pisce, jer se uvek mogu "vratiti kroz vreme" do verzije koja im je potrebna.

Primer: Zamislimo tabelu sa zalihama gde artikal ima količinu 100. Transakcija T1 (pisac) menja količinu na 80, ali još nije uradila *commit*. U tom trenutku, transakcija T2 (čitalac) pokreće upit. Firebird u lancu verzija vidi novu verziju sa vrednošću 80, ali proverom statusa transakcije T1 na TIP stranici utvrđuje da ona još nije potvrđena. Prateći *Back Interest Pointer*, čitalac T2 dolazi do prethodne verzije sa vrednošću 100 i nju isporučuje korisniku. Čim T1 uradi *commit*, svaka nova transakcija će odmah videti vrednost 80, dok će stare transakcije koje su još uvek aktivne i dalje čitati 100, čuvajući na taj način konzistentnost svog pogleda na bazu..

3.3. Copy-on-Write princip: Fizički smeštaj novih verzija podataka na stranice baze

Fizička implementacija MGA sistema u Firebirdu zasniva se na **Copy-on-Write** (kopiraj pri upisu) principu. U tradicionalnim bazama podataka, operacija `UPDATE` često podrazumeva prepisivanje starog podatka novim na istoj fizičkoj adresi na disku. U Firebirdu, međutim, originalni podatak se nikada ne menja "u mestu". Svaka modifikacija zapravo pokreće proces kreiranja nove fizičke kopije tog zapisa sa ažuriranim vrednostima, dok stara verzija ostaje netaknuta na svojoj prvobitnoj lokaciji.

Kada dođe do izmene, *engine* prvo locira slobodan prostor na stranicama podataka. Firebird teži optimizaciji tako što pokušava da novu verziju zapisa smesti na istu stranicu gde se nalazi i prethodna verzija, čime se smanjuje broj potrebnih ulazno-izlaznih (I/O) operacija prilikom čitanja lanca verzija. Ako na toj stranici nema dovoljno mesta, nova verzija se smešta na prvu sledeću slobodnu stranicu.

Ono što ovaj proces čini specifičnim jeste način na koji se održava veza između verzija. Stari zapis ostaje na svom fizičkom mestu, ali se njegovo zaglavlje ažurira tako da on postane "istorijski" zapis koji pokazuje na novu, aktuelniju verziju. Ovaj pristup ima nekoliko ključnih prednosti:

- **Brzina upisa** – baza ne gubi vreme na kompleksno pomeranje postojećih podataka ili reorganizaciju indeksa u realnom vremenu; ona jednostavno dodaje novi "vagon" na kraj lanca verzija.
- **Trenutni Rollback** – ukoliko transakcija koja je vršila izmenu propadne (Rollback), Firebird ne mora da radi nikakvu fizičku restauraciju podataka. Stari podaci su i dalje tu, netaknuti, a nova, neuspešna verzija će jednostavno biti ignorisana i kasnije uklonjena.

- **Konkurentnost** – pošto se stari podaci ne prepisuju, transakcije koje su počele pre izmene mogu neometano da čitaju originalne vrednosti bez potrebe za privremenim tabelama ili *undo* logovima.

Da bi bilo jasnije, zamislimo situaciju gde se zapis nalazi na *stranici 100*, na *slotu 5*. Kada korisnik promeni taj zapis, Firebird će na slobodnom mestu (npr. *slot 12* na istoj stranici) upisati novu verziju. Zaglavlje na *slotu 5* će sada sadržati informaciju da je taj zapis postao prethodna generacija i "uputiće" svaki novi upit na *slot 12*. Na taj način, fizička struktura baze raste sa svakom izmenom, što se kasnije rešava procesom sakupljanja otpada (*Garbage Collection*), kako bi se sprečilo nepotrebno trošenje prostora na disku (*Database Bloat*).

3.4. Transaction Inventory Page (TIP)

Da bi Firebird u svakom trenutku mogao da donese odluku o tome koju verziju zapisa sme da prikaže kojoj transakciji, on mora da se osloni na specijalizovanu strukturu pod nazivom **Transaction Inventory Page (TIP)**. Ova stranica funkcioniše kao centralni, globalni registar statusa svih transakcija koje su ikada pokrenute u bazi (ili barem onih koje još uvek imaju relevantne verzije zapisa).

Fizički, TIP je sistemska stranica koja se sastoji od niza bitova. Svakoj transakciji, čim se pokrene, dodeljuje se jedinstveni, monotono rastući redni broj – **Transaction ID (TID)**. Na TIP stranici, za svaki TID su rezervisana tačno **dva bita** koja u binarnom formatu definišu trenutni status te transakcije. Ova dva bita mogu imati četiri kombinacije od kojih svaka nosi kritičnu informaciju za mehanizam izolacije:

- **00 (Active)** – ovo je početno stanje svake transakcije. To znači da je transakcija pokrenuta, da možda trenutno vrši izmene, ali još uvek nije ni potvrđena (Commit) ni poništena (Rollback). Sve verzije zapisa koje nose ovaj TID su nevidljive za sve ostale korisnike u sistemu (osim za samu tu transakciju).
- **01 (Committed)** – ovaj status označava da je transakcija uspešno završena i da su njene izmene postale trajni deo baze. Tek kada se status na TIP stranici promeni u 01, te verzije zapisa postaju kandidati za čitanje od strane drugih, novijih transakcija.
- **10 (Rolled Back)** – ukoliko korisnik poništi transakciju ili dođe do pada aplikacije, status se postavlja na 10. Firebird ne briše odmah podatke koje je ova transakcija upisala. Umesto toga, on ih jednostavno "markira" kao nepostojeće putem ovog statusa. Prilikom sledećeg čitanja, *engine* će videti ovaj status i automatski preskočiti te verzije.
- **11 (Limbo)** — Ovo stanje se javlja isključivo kod distribuiranih transakcija (*Two-Phase Commit*). Ono signalizira da je lokalni deo posla završen, ali se čeka potvrda od spoljnog koordinatora pre nego što se transakcija konačno proglasi potvrđenom ili poništenom.

Kada upit (npr. SELECT) dođe do nekog reda na stranici podataka, on prvo "pročita" TID iz zaglavlja tog reda. Zatim, brzim pristupom u memoriju (gde se TIP stranice obično keširaju), proverava status tog TID-a. Ako je status na TIP stranici 01 (Committed), podatak se prihvata. Ako je 00 ili 10, engine prati lanac verzija unazad (preko *Back Interest* pointera) sve dok ne nađe TID koji ima status 01.

3.5. Record Header: Meta-podaci o Transaction ID-u na nivou svakog reda

Svaki zapis (red) u Firebird bazi podataka, pored korisničkih kolona koje definišu šemu tabele, sadrži i fiksni, nevidljivi deo koji se naziva **Record Header** (zaglavlje zapisa). Ovo zaglavlje predstavlja kontrolni centar za MGA mehanizam, jer upravo u njemu *engine* baze pronalazi ključne metapodatke neophodne za navigaciju kroz verzije i određivanje vidljivosti podataka u višekorisničkom okruženju.

Struktura zaglavlja zapisa u Firebirdu sadrži nekoliko elemenata koji omogućavaju funkcionisanje MVCC modela:

- **Transaction ID (TID) – p**redstavlja redni broj transakcije koja je kreirala tu specifičnu verziju reda. Ovaj identifikator je direktno povezan sa TIP stranicom; kada engine pročita TID iz zaglavlja, on odmah "zna" koju poziciju na TIP stranici treba da proveri kako bi utvrdio da li je ta verzija potvrđena, aktivna ili poništena.
- **Back Interest Pointer –** ovo je fizički pokazivač koji vodi do prethodne verzije tog istog reda na stranicama podataka. Ukoliko trenutna verzija zapisa nije validna za transakciju koja vrši čitanje (npr. jer je suviše "mlada" ili još uvek aktivna), engine koristi ovaj pokazivač kako bi se vratio jedan korak unazad u istoriji izmena. Na taj način se formira lanac verzija koji omogućava rekonstrukciju stanja baze za bilo koju aktivnu transakciju.
- **Record Flags – s**kup bitova koji definišu stanje samog zapisa na stranici, kao što su informacije o tome da li je zapis fragmentisan (ako ne može da stane na jednu stranicu) ili da li predstavlja najnoviju verziju u lancu.

Zahvaljujući ovim metapodacima, Firebird može veoma brzo da donese odluku o isporuci podataka korisniku bez potrebe za eksternim tabelama zaključavanja. Kada upit stigne do određenog fizičkog slota na stranici, on prvo analizira **TID** iz zaglavlja. Upoređivanjem tog broja sa stanjem na **TIP stranici** i uzimajući u obzir nivo izolacije u kojem transakcija radi (npr. *Snapshot* ili *Read Committed*), engine momentalno odlučuje: ili se podatak čita direktno sa te lokacije, ili se prati **Back Interest Pointer** ka starijoj generaciji.

Primer: Ukoliko transakcija sa brojem 600 pokuša da pročita red čije zaglavlje nosi TID 605, engine će uvideti da je taj podatak kreirala transakcija koja je počela nakon nje. Bez obzira na to da li je transakcija 605 već uradila *Commit*, transakcija 600 je "ne sme" videti u *Snapshot* režimu. Engine tada koristi pokazivač iz zaglavlja verzije 605, "skače" na lokaciju prethodne verzije (npr. one sa TID 580) i proverava njenu vidljivost. Ovaj proces se odvija transparentno i munjevito, osiguravajući da svaki korisnik uvek ima konzistentan pogled na podatke, bez obzira na intenzitet izmena koje vrše drugi učesnici u sistemu.

4. Implementacija nivoa izolacije u Firebird-u

Izolacija transakcija u Firebirdu nije rigidna, fiksna kategorija, već fleksibilan sistem koji se prilagođava specifičnim potrebama poslovnih procesa. Dok određeni procesi zahtevaju apsolutnu digitalnu preciznost i nepoverljivost prema paralelnim izmenama (poput izrade završnog računa), drugi primarno zahtevaju uvid u najsvežije dostupne podatke u realnom vremenu (poput kontrole zaliha u maloprodaji). Firebird ove varijabilne zahteve rešava kroz tri osnovna nivoa izolacije, koji direktno upravljaju načinom na koji engine baze interpretira lance verzija zapisa.

4.1. Snapshot

Snapshot je nivo izolacije koji korisniku pruža "zamrznutu sliku" baze podataka, osiguravajući potpuno dosledan pogled na podatke u trenutku startovanja transakcije. Onog momenta kada transakcija započne rad, Firebird u njenom kontekstu beleži stanje TIP stranice, odnosno identifikuje broj poslednje potvrđene transakcije. Od tog trenutka, pa sve do samog završetka, transakcija živi u svojevrsnom izolovanom vremenskom mehuru — ona vidi bazu tačno onakvu kakva je bila u sekundi njenog pokretanja.

Čak i ako desetine drugih korisnika u međuvremenu izmene i potvrde milione zapisa, Snapshot transakcija će te promene ignorisati. Ona će za svaki traženi red uporno pratiti lanac verzija unazad, sve dok ne pronađe generaciju koja je bila potvrđena pre njenog "rođenja". Ovakav pristup je nezamenljiv kod dugačkih i kompleksnih izveštaja koji zahtevaju unutrašnju logičku konzistentnost. Na primer, ukoliko se štampa lista svih faktura uz njihov kumulativni zbir, Snapshot izolacija garantuje da će suma do poslednje decimale odgovarati prikazanim stavkama, bez obzira na to što paralelni procesi u tom trenutku možda dodaju nove fakture ili menjaju postojeće.

4.2. Read Committed

Za razliku od statične prirode Snapshot-a, Read Committed nivo je znatno dinamičniji i prilagođeniji transakcionim sistemima visoke frekvencije. Ovaj nivo dozvoljava transakciji da "vidi" promene koje su druge transakcije potvrdile čak i nakon što je ona sama počela sa radom. Na taj način se postiže optimalan balans između ažurnosti informacija i performansi sistema. Kad god se izvrši novi SQL upit unutar ovakve transakcije, Firebird ponovo konsultuje TIP stranicu i isporučuje najnovije potvrđene verzije zapisa koje su dostupne u tom trenutku.

4.2.1. Razlika između RECORD VERSION i NO RECORD VERSION

Unutar Read Committed nivoa, Firebird nudi dve ključne pod-varijante koje suštinski menjaju ponašanje baze u situacijama kada dođe do pokušaja pristupa podacima koji se trenutno menjaju:

- **RECORD VERSION** – ukoliko transakcija A pokuša da pročita red nad kojim transakcija B trenutno vrši izmenu, Firebird neće zaustaviti proces. Umesto čekanja, on će trenutno "posegnuti" u lanac verzija i isporučiti poslednju potvrđenu generaciju tog reda. Ovo omogućava maksimalnu propusnu moć sistema jer čitaoci nikada ne čekaju na pisce.
- **NO RECORD VERSION** – u ovom režimu, ako je red koji korisnik želi da pročita u procesu izmene (odnosno postoji njegova novija, još uvek nepotvrđena verzija), transakcija će pauzirati svoj rad. Ona ulazi u stanje čekanja sve dok se ta druga transakcija ne završi

(Commit ili Rollback). Tek po razrešenju konflikta, korisnik dobija najnoviji, upravo potvrđeni podatak. Ovaj režim se koristi u kritičnim scenarijima gde je imperativ raditi sa apsolutno poslednjom verzijom podatka, svesno prihvatajući kratkotrajna zastoja u radu.

4.3. Snapshot Table Stability

Snapshot Table Stability (poznat i kao *Consistency*) predstavlja najstroži i najekskluzivniji režim izolacije u Firebird-u, koji je funkcionalni ekvivalent Serializable nivou iz SQL standarda. Kada transakcija zahteva ovaj nivo, ona praktično traži ekskluzivni monopol nad tabelama kojima pristupa. Ovaj režim sprečava bilo koju drugu transakciju da vrši izmene u tabelama koje je Snapshot Table Stability transakcija "rezervisala" za svoj rad.

Iako pruža najviši stepen sigurnosti, ovaj nivo se u modernom višekorisničkom razvoju koristi ekstremno retko i uz veliki oprez. Njegova primena može dovesti do potpune paralize sistema jer blokira rad svih ostalih korisnika nad tim tabelama. Obično se čuva za masovne administrativne operacije, migracije podataka ili kompleksne proračune gde je bilo kakva paralelna aktivnost apsolutno nedopustiva.

4.4. Read Committed Read Consistency

Jedan od najvećih izazova u starijim verzijama Firebirda bio je problem "izgubljenih ažuriranja" (update conflicts) u Read Committed modu. Firebird 4.0 je uveo revolucionarni režim pod nazivom READ CONSISTENCY.

U ovom režimu, svaki pojedinačni SQL operacija (na primer, jedan UPDATE koji menja hiljade redova istovremeno) dobija svoj sopstveni, privremeni mini-snapshot. To osigurava da je svaka instrukcija konzistentna sama sa sobom tokom celog svog trajanja. Ukoliko tokom izvršavanja tog upita neka druga transakcija potvrdi izmene na istim podacima, Firebird neće prekinuti rad greškom "Deadlock" ili "Update Conflict". Umesto toga, on će inteligentno i automatski "ponoviti" deo upita kako bi osigurao da su sve promene ispravno primenjene na najnovije, upravo potvrđeno stanje baze. Ovo predstavlja ogroman iskorak za stabilnost aplikacija jer baza preuzima na sebe kompleksan posao sinhronizacije koji su ranije morali ručno da rešavaju programeri kroz komplikovane mehanizme ponavljanja transakcija (*retry logic*).

.

5. Mehanizmi zaključavanja (locking) i rešavanje konflikata

Iako arhitektura Firebirda omogućava čitaocima da nikada ne blokiraju pisce, situacija se drastično menja u momentu kada dve ili više transakcija pokušaju da istovremeno izvrše izmenu nad istim logičkim redom podataka. U takvim situacijama, MGA mehanizam verzija više nije dovoljan da osigura integritet, te na scenu stupaju mehanizmi zaključavanja. Ovi mehanizmi garantuju da u svakom trenutku samo jedna transakcija može imati status "vlasnika" izmene nad određenim resursom, čime se sprečava gubitak podataka i narušavanje poslovne logike.

5.1. Pesimistička kontrola

U specifičnim poslovnim procesima, programer često ne želi da čeka trenutak fizičkog upisa (UPDATE) da bi saznao da li je određeni podatak slobodan za manipulaciju. Postoje situacije gde je neophodno "rezervisati" red odmah pri samom čitanju, pre nego što korisnik uopšte započne unos novih vrednosti. Za tu svrhu Firebird nudi klauzulu `SELECT ... WITH LOCK`.

Ovaj pristup se naziva **pesimističko zaključavanje** jer se zasniva na pretpostavci da će verovatno doći do konflikta, pa se resurs preventivno obezbeđuje. Čim se izvrši ovakav upit, Firebird postavlja ekskluzivni katanac na pročitane redove. Svaka druga transakcija koja pokuša da izvrši **UPDATE** ili **DELETE** nad tim redovima biće trenutno blokirana ili odbijena (zavisno od parametara čekanja) sve dok se prva transakcija ne završi putem *commit* ili *rollback* naredbe.

Primer: U sistemima za hotelske rezervacije ili prodaju avio-karata, ključno je da u momentu kada korisnik odabere slobodno mesto, niko drugi ne može da mu ga "otme" dok on popunjava svoje lične podatke na ekranu. Upotrebom **SELECT WITH LOCK**, aplikacija rezerviše to sedište u bazi istog trenutka kada ga prikaže korisniku, čime se izbegava situacija da dva putnika plate istu kartu.

5.2. Optimistički pristup

Standardni i podrazumevani način rada Firebirda je **optimistička kontrola**. Ovde baza dozvoljava korisniku da slobodno čita podatke bez ikakvog zaključavanja, pod pretpostavkom da je verovatnoća konflikta mala. Međutim, prava provera se vrši u "zadnji čas" — u momentu kada korisnik pokuša da izvrši **UPDATE**.

Ukoliko je u intervalu između čitanja i pokušaja upisa neka druga transakcija već izmenila i potvrdila (*commit*) taj isti red, Firebird će prvoj transakciji izbaciti grešku poznatu kao ***Update Conflict***. Sistem praktično signalizira: „Verzija zapisa koju ti držiš u memoriji više nije najnovija verzija u bazi“. Programer u ovom slučaju mora predvideti obradu izuzetaka u kodu — najčešće tako što će ponovo učitati sveže podatke, prikazati ih korisniku uz informaciju o tuđoj izmeni, ili pokušati automatsko spajanje promena (*merge logic*).

5.3. Arhitektura i uloga Lock Manager-a

Za preciznu koordinaciju svih katanaca, provera i statusa je zadužen **Lock Manager**. To je mehanizam koji vodi rigoroznu evidenciju o svim aktivnim zaključavanjima na nivou celog servera. Njegova implementacija zavisi od arhitekture servera npr. kod *SuperServer* modela, on je integrisan unutar samog procesa, dok kod *Classic* ili *SuperClassic* modela funkcioniše kao poseban entitet ili kroz deljenu memoriju.

Lock Manager koristi **deljenu memoriju (engl. Shared Memory)** kako bi u realnom vremenu pratio ko drži koji katanac i koji procesi čekaju na red. On ne upravlja samo zaključavanjem redova, već i:

- **Zaključavanjem tabela** – kada se vrše operacije nad strukturom (DDL).
- **Zaključavanjem stranica baze** – prilikom fizičkih operacija upisa na disk.
- **Logičkim objektima** – poput generatora (sekvenci) i eksternih datoteka.

Njegova uloga je uloga "saobraćajca" koji održava red u višekorisničkom okruženju. Efikasnost Lock Manager-a je presudna za skalabilnost – ako je loša konfiguracija parametara zaključavanja onda će se on pretvoriti u „usko grlo” pri velikom broju konkurentnih sesija.

5.4. Deadlock u Firebirdu: Algoritmi detekcije i strategije izbegavanja

Deadlock, iliti mrtva petlja, je klasičan problem paralelnog programiranja koji se javlja kada dve ili više transakcija uđu u začarani krug čekanja. Na primer, *transakcija A* zaključa *red 1* i čeka da se oslobodi *red 2*, dok istovremeno *transakcija B* drži Red 2 i čeka na Red 1. U tom trenutku, oba procesa ostaju trajno zamrznuta jer niko ne može da nastavi rad.

Firebird poseduje sofisticiran ugrađen algoritam za detekciju ovakvih situacija. Lock Manager periodično analizira **graf čekanja (engl. *wait-for graph*)**. Čim detektuje postojanje zatvorene petlje, on interveniše tako što nasilno prekida jednu od transakcija u krugu i šalje joj grešku ***„Deadlock“***. Time se oslobađaju resursi i omogućava drugoj transakciji da završi svoj posao. Najbolja strategija za programere je uvek ista: ažuriranje tabela uvek treba vršiti istim redosledom u svim delovima aplikacije, čime se matematički minimizuje mogućnost nastanka kružnog čekanja.

5.5. Parametri čekanja: Wait, No Wait i uticaj Lock Timeout parametra

Kada jedna transakcija naiđe na red koji je već zaključan od strane druge transakcije, njeno dalje ponašanje direktno zavisi od parametara definisanih unutar **TPB (Transaction Parameter Block)** bloka prilikom njenog startovanja:

- **WAIT** – transakcija ulazi u stanje mirovanja i čekaće onoliko dugo koliko je potrebno da se zaključani red oslobodi. Iako ovo deluje logično, u praksi može dovesti do "zamrzavanja" korisničkog interfejsa ako druga transakcija ostane otvorena predugo (npr. korisnik je otišao na pauzu dok mu je otvoren prozor za izmenu).
- **NO WAIT** – transakcija se momentalno odbija uz grešku ukoliko naiđe na bilo kakav katanac. Ovo je idealno za interaktivne aplikacije (GUI) jer korisnik trenutno dobija povratnu informaciju da je podatak zauzet i može nastaviti sa drugim poslom.
- **LOCK TIMEOUT** – predstavlja moderan kompromis uveden u novijim verzijama Firebirda. Transakcija će čekati definisani broj sekundi (npr. 5 ili 10). Ukoliko se u tom

roku red ne oslobodi, transakcija prekida čekanje i podiže izuzetak. Ovo sprečava beskonačno čekanje i omogućava sistemu da ostane operativan čak i u ekstremnim situacijama kada konekcije "pucaju" dok drže aktivne lock-ove.

6. Održavanje baze i garbage collection

Specifičnost Firebird arhitekture, zasnovane na MGA modelu, ogleda se u tome što sistem nikada fizički ne briše podatke u trenutku izvršenja komande. Svaka UPDATE ili DELETE operacija za sobom ostavlja staru verziju zapisa (tzv. *back version*). Ukoliko se ovi "ostaci" ne uklanjaju sistematski, baza podataka se suočava sa degradacijom performansi. Proces identifikacije i trajnog uklanjanja ovih nepotrebnih generacija podataka naziva se **Garbage Collection (GC)**.

6.1. Problem rasta baze podataka (Database Bloat) usled akumulacije verzija zapisa

U transakcionim sistemima sa visokim intenzitetom izmena, Firebird može generisati hiljade novih verzija zapisa u veoma kratkom vremenskom intervalu. Ako transakcije traju predugo, te verzije moraju ostati fizički prisutne na stranicama podataka jer postoji teoretska šansa da su nekoj od aktivnih transakcija one još uvek potrebne za održavanje konzistentnosti pogleda.

Rezultat ovog procesa je pojava poznata kao **Database Bloat**. Ova pojava znači da fizička veličina fajla baze podataka raste, dok performanse eksponencijalno opadaju. *Engine* je primoran da prilikom svakog upita pretražuje i preskače čitave nizove "mrtvih" verzija kako bi rekonstruisao aktuelno stanje, što troši dragocene procesorske resurse i povećava broj I/O operacija. Razumevanje mehanizama čišćenja je stoga imperativ za svakog administratora ili programera koji radi sa Firebirdom.

Firebird implementira tri različite strategije za neutralisanje nepotrebnih verzija zapisa, prilagođene različitim scenarijima opterećenja:

- **Cooperative Garbage Collection –** ovo je najrasprostranjeniji vid čišćenja koji se odvija "reaktivno". Dešava se tokom redovnog rada, kada korisnički SELECT upit naiđe na lanac verzija. Engine, dok čita podatke, vrši proveru: ako identifikuje verzije koje su starije od najstarije aktivne transakcije, on ih odmah uklanja. Na ovaj način, sami korisnici kroz svoje upite doprinose održavanju higijene baze podataka.
- **Background Garbage Collection** – karakterističan za modernije verzije (posebno u *SuperServer* modelu), ovaj mehanizam koristi posebnu programsku nit (thread) koja u pozadini, nezavisno od korisničkih aktivnosti, skenira stranice i vrši čišćenje. Time se primarni korisnički upiti rasterećuju posla oko održavanja lanca verzija.
- **Sweeping (Backsweep)** – ovo je najtemeljniji process i on predstavlja sistematsko skeniranje celokupne baze podataka. *Sweep* identifikuje sve nepotrebne verzije, oslobađa prostor i ažurira status na TIP stranicama. Može se konfigurisati da se pokreće automatski (nakon definisanog broja transakcija, tzv. *Sweep Interval*) ili ručno putem gfix alata. Iako privremeno usporava bazu, neophodan je za "resetovanje" indikatora zdravlja baze.

6.2. Analiza transakcionih markera

Rad *Garbage Collector* mehanizma i opšte performanse Firebird baze podataka direktno zavise od četiri ključna transakciona markera. Ovi parametri se čuvaju u zaglavlju baze podataka (*Database Header*) i predstavljaju kontrolne tačke koje *engine* koristi za navigaciju kroz verzije zapisa. Razumevanje ovih vrednosti je od presudnog značaja za dijagnostiku „zagušenja" baze i prekomernog rasta fizičkog fajla.

6.2.1. Definicija markera i njihova uloga

Tipovi markera su sledeći:

- **Next Transaction ID** – predstavlja redni broj (identifikator) koji će biti dodeljen prvoj sledećoj transakciji koja se pokrene. Ovaj broj se konstantno uvećava i služi kao gornja granica hronologije rada baze.
- **Oldest Interesting Transaction (OIT)** – označava najstariju transakciju koja sistemu još uvek predstavlja „interes". To je najstariji ID transakcije koja je ili još uvek aktivna, ili je poništena (*Rolled Back*), ali njene verzije zapisa još uvek nisu fizički očišćene sa stranica baze podataka. OIT je donji prag ispod kojeg čišćenje više ne može da ide.
- **Oldest Active Transaction (OAT)** – predstavlja ID najstarije transakcije koja se trenutno nalazi u statusu *Active*. Ovo je najkritičniji marker jer direktno blokira proces sakupljanja otpada. *Garbage Collector* ne sme da ukloni nijednu verziju zapisa koja je nastala nakon OAT-a, jer ta transakcija potencijalno može zahtevati pristup tim podacima u skladu sa svojim nivoom izolacije.
- **Oldest Snapshot Transaction (OST)** – ovaj marker identifikuje najstariju transakciju pokrenutu u *Snapshot* nivou izolacije. Specifičnost ovog markera je u tome što on „zamrzava" stanje cele baze za tu transakciju, sprečavajući uklanjanje svih verzija koje su postojale u trenutku njenog starta.

Zdravstveno stanje baze se meri razlikom (*Gap*) između markera. U idealnim uslovima, ovi brojevi su blizu jedan drugom. Vrednosti se mogu videti komandom `gstat -h.`

- **Primer 1 (Zdrava baza)** – Next: 10500, OAT: 10490, OIT: 10485 Razlika je mala, što znači da se transakcije brzo završavaju i da *Garbage Collector* uspešno prati tempo rada.
- **Primer 2 (Problematična baza - Stuck Transaction)** – Next: 25000, OAT: 5000, OIT: 4950 Ovde postoji ogroman jaz. Transakcija ID 5000 je otvorena pre 20.000 ciklusa i još nije zatvorena. Sve verzije podataka nastale u tom međuvremenu se nagomilavaju jer ih baza ne sme obrisati. Ovo dovodi do drastičnog pada performansi SELECT upita.

6.2.2. SQL monitoring markera u realnom vremenu

Uvođenjem monitoring tabela (od verzije 2.1), Firebird je omogućio administratorima da vrše inspekciju unutrašnjeg stanja baze podataka koristeći standardnu SQL sintaksu. Ovi podaci se ne čitaju sa diska, već se generišu u realnom vremenu iz kontrolnih struktura u memoriji servera..

Primer 1: Upit za analizu transakcionog „jaza”

Ovaj upit je primarni alat za proveru "zdravlja" baze. Njegov cilj je da identifikuje razliku između najnovije transakcije i onih koje još uvek drže resurse.

Izvorni kod: Upit za analizu transakcionog „jaza”

```
1  /* ================================================
2     Pregled transakcionih markera – zdravlje baze
3     ================================================ */
4  +
5  SELECT
6      MON$NEXT_TRANSACTION AS NEXT_ID,  -- Sledeći ID koji će biti
   dodeljen transakciji
7      MON$OLDEST_TRANSACTION AS OIT,    -- Najstarija interesantna
   transakcija (donja granica čišćenja)
8      MON$OLDEST_ACTIVE AS OAT,         -- Najstarija AKTIVNA
   transakcija – BLOKIRA garbage collection!
9      MON$OLDEST_SNAPSHOT AS OST,       -- Najstarija snapshot
   transakcija (drži stare verzije)
10     (MON$NEXT_TRANSACTION - MON$OLDEST_ACTIVE) AS TRANSACTION_GAP
   -- Broj verzija koje GC ne može da očisti
11 FROM MON$DATABASE;
12
13 /* ================================================
14    TUMAČENJE:
15
16    OAT (Oldest Active Transaction):
17       - Najkritičniji marker
18       - Sve transakcije STARIJE od OAT-a mogu da se čiste
19       - Sve novije od OAT-a moraju da se čuvaju
20
21    TRANSACTION_GAP:
22       - < 1000  → zdrava baza
23       - 1000-10000 → umeren rizik
24       - > 10000 → predstavlja ozbiljan problem jer će baza biti
   sve sporija
25
26    OIT (Oldest Interesting Transaction):
27       - Najstarija transakcija čije verzije još uvek postoje
28
29    OST (Oldest Snapshot):
```

```
      - Ako je mnogo manji od NEXT_ID, neka snapshot transakcija
        "drži" stare verzije i sprečava čišćenje

   PRIMER ZDRAVE BAZE:
   NEXT_ID = 15000, OAT = 14950, OIT = 14948, GAP = 50

   PRIMER PROBLEMATIČNE BAZE:
   NEXT_ID = 50000, OAT = 10000, OIT = 9995, GAP = 40000
   – transakcija ID 10000 je otvorena pre 40000 ciklusa!
   ================================================ */
```

Primer 2: Upit za identifikaciju blokirajućih procesa

Kada prethodni upit pokaže veliki jaz, neophodno je identifikovati konkretnu sesiju koja ga uzrokuje. Sledeći upit povezuje transakcije sa sesijama kako bi se dobili podaci o fizičkom korisniku.

Izvorni kod: Upit za identifikaciju blokirajuće transakcije

```
/* ================================================
   IDENTIFIKACIJA TRANSAKCIJE KOJA BLOKIRA GARBAGE COLLECTOR
   ================================================ */
SELECT
    A.MON$USER AS KORISNIK,
    A.MON$REMOTE_ADDRESS AS IP_ADRESA,
    A.MON$REMOTE_PROCESS AS APLIKACIJA,
    T.MON$TIMESTAMP AS POCETAK,
    (CURRENT_TIMESTAMP - T.MON$TIMESTAMP) AS TRAJANJE_SEKUNDI,
    CASE T.MON$ISOLATION_MODE
        WHEN 0 THEN 'Consistency'
        WHEN 1 THEN 'Snapshot'
        WHEN 2 THEN 'RC Record Version'
        WHEN 3 THEN 'RC No Record Version'
        WHEN 4 THEN 'Read Consistency'
        ELSE 'Nepoznat'
    END AS IZOLACIJA
FROM MON$TRANSACTIONS T
JOIN MON$ATTACHMENTS A ON T.MON$ATTACHMENT_ID =
A.MON$ATTACHMENT_ID
WHERE T.MON$TRANSACTION_ID = (SELECT MON$OLDEST_ACTIVE FROM
MON$DATABASE);
/* ================================================
      TUMAČENJE:
       Ovaj upit pronalazi transakcije zbog kojih Garbage
Collector ne može da čisti stare verzije
      MON$OLDEST_ACTIVE (OAT) = najstarija aktivna transakcija,
što je OAT stariji, više verzija se gomila = BLOAT
```

```

      Ako TIME_START pokazuje da transakcija traje:
      - > 5 minuta = problem
   ================================================ */
```

U tablici je dat pregled potencijalnih vrednosti za nivo izolacije i mali podsednik na objašnjenje za svaku od njih:

Vrednost	Izolacija	Objašnjenje
0	Consistency	Najstroža izolacija – vidi samo stanje baze na početku transakcije
1	Concurrency (Snapshot)	Standardni **snapshot** – najčešće korišćen
2	Read Committed (Record Version)	Čita poslednju commitovanu verziju reda
3	Read Committed (No Record Version)	Čeka ako je red zaključan
4	Read Committed Read Consistency	Noviji snapshot za read committed

Tabela 2: Vrednosti MON$ISOLATION_MODE

Analiza putem MON$ tabela nije samo informativna; ona direktno oslikava (ne)efikasnost Garbage Collector-a. U Firebird arhitekturi, GC konstantno prati vrednost *MON$OLDEST_ACTIVE* (OAT). Sve verzije zapisa čiji je Transaction ID manji od OAT-a, a veći od OIT-a, postaju primarna meta za čišćenje.

Međutim, čim SQL monitoring pokaže da TRANSACTION_GAP raste, to je direktan dokaz da je Garbage Collector "blokiran". On fizički ne može da pristupi stranicama baze i oslobodi prostor jer mu pravila MGA arhitekture nalažu da čuva sve verzije koje bi najstarija aktivna transakcija) potencijalno mogla da zatraži. Dakle, SQL monitoring nam omogućava da vidimo "uskim grlima" GC-a pre nego što ona dovedu do kritičnog pada performansi.

7. Planovi izvršavanja (Execution Plans) i monitoring

Čak i savršeno indeksirana baza može pokazati drastičan pad performansi ako **Optimizer** (podsistem zadužen za putanju do podataka) donese odluku koja ne uzima u obzir trenutno transakciono stanje sistema. U Firebirdu, planovi izvršavanja nisu statični; oni su duboko povezani sa stanjem verzija zapisa i nivoom izolacije u kojem se upit pokreće.

7.1. Uloga TPB (Transaction Parameter Block) u donošenju odluka

Svaka interakcija između klijentske aplikacije (npr. Delphi aplikacije preko FireDAC drajvera) i servera započinje definisanjem **TPB** strukture. Ovo je binarni protokol kojim aplikacija diktira uslove pod kojima će se upiti izvršavati. Optimizer koristi ove parametre kao primarne ulazne podatke za sastavljanje plana:

- **Access Mode (Read Only vs. Read Write):** Ukoliko je transakcija deklarisana kao *Read Only*, Optimizer eliminiše sve rutine koje se tiču zaključavanja i pripreme za potencijalni upis, što značajno rasterećuje CPU.
- **Isolation Level:** Ako je nivo izolacije *Snapshot Table Stability*, Optimizer dobija informaciju da se podaci u tabelama neće menjati dok transakcija traje. To mu omogućava da planira pristupe bez straha od paralelnih izmena, često birajući agresivnije algoritme za spajanje tabela (*Merge Joins*).

7.2. Uticaj nivoa izolacije na izbor putanje: Indeksi vs. Natural Scan

U Firebird arhitekturi, indeksi su takođe podložni MGA mehanizmu. Svaki ključ u indeksu mora biti proveren naspram stanja vidljivosti transakcije. Ovde nastaje korelacija između "prljave" baze i sporih upita:

1. **Problem akumuliranih verzija:** Ako tabela ima visok nivo *bloat-a* (zbog niske vrednosti OAT markera), indeksi postaju pretrpani pokazivačima na verzije koje više nisu validne, ali još nisu očišćene.
2. **Snapshot kazna:** U *Snapshot* izolaciji, upit koji koristi indeks nad tabelom sa mnogo verzija može biti sporiji nego što se očekuje. Razlog je taj što engine baze mora za svaki ključ u indeksu da izvrši proveru na **TIP stranici** kako bi utvrdio da li je ta konkretna verzija vidljiva.
3. **Povlačenje u "Natural Scan":** U ekstremnim slučajevima, kada Optimizer detektuje da je broj nevalidnih verzija u indeksu previsok, on može doneti odluku da potpuno ignoriše indeks i uradi **NATURAL scan** (čitanje cele tabele). Ovo objašnjava situacije gde isti upit radi trenutno na test bazi, dok u produkciji sa mnogo korisnika "zamrzava" aplikaciju.

7.3. Dijagnostika transakcija u realnom vremenu putem sistemskih tabela (MON$TRANSACTIONS)

Za razliku od prethodno opisanih monitoringa markera, ovde nas zanima šta se tačno dešava "ispod haube" dok se upit izvršava. Tabela **MON$STATEMENTS** nam omogućava da vidimo plan koji je Optimizer izabrao za svaki upit koji je trenutno u toku.

Primer: SQL primer za detekciju sporih upita i njihovih planova

Ovaj upit direktno povezuje korisnika, njegov nivo izolacije i plan koji baza trenutno koristi:

Izvorni kod: SQL primer za detekciju aktivnih SQL upita sortiranih u opadajućem redosledu po brzini

```
/* ================================================
   DETEKCIJA AKTIVNIH SQL UPITA
   ================================================ */

SELECT
    A.MON$USER AS KORISNIK,                          -- Ko izvršava upit
    ST.MON$SQL_TEXT AS SQL_UPIT,                      -- Tekst SQL upita koji se izvršava
    ST.MON$TIMESTAMP AS POCETAK,                      -- Vreme početka izvršavanja
    (CURRENT_TIMESTAMP - ST.MON$TIMESTAMP) AS TRAJANJE_SEKUNDI, -- Koliko dugo traje
    T.MON$ISOLATION_MODE AS IZOLACIJA_ID,             -- Nivo izolacije (sirovi broj)
    T.MON$TRANSACTION_ID AS TID                        -- ID transakcije kojoj upit pripada
FROM MON$STATEMENTS ST
JOIN MON$TRANSACTIONS T ON ST.MON$TRANSACTION_ID = T.MON$TRANSACTION_ID
JOIN MON$ATTACHMENTS A ON T.MON$ATTACHMENT_ID = A.MON$ATTACHMENT_ID
WHERE ST.MON$STATE = 1                                 -- 1 = aktivan upit
ORDER BY TRAJANJE_SEKUNDI DESC;                        -- Najsporiji prvi

/* ================================================
   TUMAČENJE:
   - Prikazuje SQL upite koji se UPRAVO izvršavaju
   - TRAJANJE_SEKUNDI pokazuje potencijalno spore upite
   ================================================ */
```

.

7.4. Zašto je Snapshot sporiji kod "dugih" transakcija?

Treba istaći da *Snapshot* izolacija zahteva više resursa od *Read Committed* režima. Kod *Snapshot* transakcije, *engine* mora da prati sve *Back Interest* pokazivače (iz poglavlja 3.5) kako bi rekonstruisao stanje od pre tri ili četiri sata. Što je baza aktivnija, ovi lanci su duži, a *Optimizer*-ov posao postaje teži. Zato se u visokofrekventnim sistemima (poput ERP rešenja ili e-commerce platformi) preporučuje korišćenje *Read Committed* režima sa *Record Version* pod-varijantom za većinu svakodnevnih operacija, dok se *Snapshot* čuva isključivo za specifične izveštaje.

8. Praktična demonstracija i studija slučaja

U ovom poglavlju sprovodimo seriju testova nad bazom **EMPLOYEE.FDB** kako bismo praktično dokazali anomalije, nivoe izolacije i unutrašnje mehanizme baze.

Primer 1: Dokaz o sprečavanju anomalije "Prljavo čitanje (Dirty Read)"

Opis: Firebird po dizajnu ne dozvoljava čitanje nepotvrđenih podataka. U ovom testu Sesija A menja podatak, a Sesija B pokušava da ga očita pre potvrde.

Rezultat: Sesija B očitava staru vrednost = 105900.00, što nam dokazuje da Firebird sprečava anomaliju **„Dirty Read"** bez obzira na podešavanja, čime garantuje konzistentnost

Izvorni kod: Nemogućnost anomalije "Prljavo čitanje"

```
/* ================================================
   Primer 1: Dokaz o sprečavanju anomalije "Prljavo čitanje"

   Sesija A (prvi prozor) – pokreće izmenu bez potvrde
   ------------------------------------------------- */
UPDATE EMPLOYEE
SET SALARY = 120000
WHERE EMP_NO = 2;

/* -- NE RADIMO COMMIT! Transakcija ostaje otvorena --

   Sesija B (drugi prozor, paralelno) – pokušava čitanje
   ------------------------------------------------- */
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SELECT SALARY
FROM EMPLOYEE
WHERE EMP_NO = 2;

/* ================================================
   REZULTAT:

   Iako je Sesija A izmenila platu na 120000,
```

```
   Sesija B i dalje vidi STARU vrednost (npr. 105900).

   Ovo DOKAZUJE da Firebird NE DOZVOLJAVA prljavo čitanje
   – nijedna transakcija ne može videti nepotvrđene izmene!
   ================================================ */
```

Primer 2: Demonstracija anomalije "Neponovljivo čitanje (Non-repeatable Read)"

Opis: Testiramo **READ COMMITTED** nivo gde se unutar sesija A podaci mogu promeniti unutar nje ako druga sesija B izvrši COMMIT. Sekcije A i B treba da budu u dva odvojena prozora

Rezultat: Vrednost unutar transakcije A je promenjena što potvrđuje da READ COMMITTED uvek čita najnoviju potvrđenu verziju reda.

Izvorni kod: Anomalija "Neponovljivo čitanje"

```
/* ================================================
   Primer 2: Demonstracija anomalije "Neponovljivo čitanje"
   ================================================

   Sesija A (prvi prozor) – pokreće READ COMMITTED transakciju
   ------------------------------------------------ */
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
/* Prvo čitanje – vidimo trenutnu vrednost */
SELECT SALARY
FROM EMPLOYEE
WHERE EMP_NO = 2;
/* ------------------------------------------------
   Sesija B (drugi prozor) – menja podatak i potvrđuje
   ------------------------------------------------ */
UPDATE EMPLOYEE
SET SALARY = 120000
WHERE EMP_NO = 2;

COMMIT;  -- Potvrđujemo izmenu

/* ------------------------------------------------
   Sesija A (prvi prozor) – ponovo čita ISTI red
   ------------------------------------------------ */
SELECT SALARY
FROM EMPLOYEE
WHERE EMP_NO = 2;

/* ================================================
   REZULTAT:
   Prvi SELECT: 105900.00  (stara vrednost)
```

```
   Drugi SELECT: 120000.00  (nova vrednost)

   Iako je Sesija A i dalje u ISTOJ transakciji,
   vrednost se promenila tj. došlo je do
   anomalije NEPONOVLJIVO ČITANJE.
   ================================================ */
```

Primer 3: Potpuna izolacija u „Snapshot“ nivou i sprečavanje pojave fantomskog čitanja (Phantom Read)

Opis: Testiramo stabilnost pogleda na bazu. Snapshot transakcija ne sme videti nove redove ubačene nakon njenog početka Sesija A pokreće SNAPSHOT transakciju i broji redove u tabeli EMPLOYE. Sesija B ubacuje novog zaposlenog i potvrđuje INSERT.

Rezultat: Sesija A u oba slučaja vraća 104 reda, iako u bazi fizički postoji 105 reda. Ovo potvrđuje stabilnost verzije podataka koju Snapshot čuva, ali i to da Snapshot pokriva anomaliju “fantomsko čitanje”.

Izvorni kod: Snapshot i “fantomsko čitanje”

```
/* ================================================
   Primer 3: Snapshot izolacija i sprečavanje "fantomskog čitanja"

   Sesija A (prvi prozor) – pokreće SNAPSHOT transakciju
   ------------------------------------------------ */
SET TRANSACTION ISOLATION LEVEL SNAPSHOT;

/* Prvo brojanje – pamtimo trenutni broj redova */
SELECT COUNT(*) AS UKUPNO_ZAPOSLENIH
FROM EMPLOYEE;

/* NE RADIMO COMMIT - Transakcija ostaje otvorena

   Sesija B (drugi prozor) – ubacuje NOVOG zaposlenog
   ------------------------------------------------ */
INSERT INTO EMPLOYEE (EMP_NO, FIRST_NAME, LAST_NAME, PHONE_EXT,
                      HIRE_DATE, DEPT_NO, JOB_CODE, JOB_GRADE,
                      JOB_COUNTRY, SALARY)
VALUES (999, 'Phantom', 'Read', '233','28-DEC-1988', '621', 'Eng',
   2, 'USA', 97500);
COMMIT;  -- Potvrđujemo unos novog zaposlenog

/* ------------------------------------------------
   Sesija A (prvi prozor) – ponovo brojimo redove
   ------------------------------------------------ */
SELECT COUNT(*) AS UKUPNO_ZAPOSLENIH
```

```
FROM EMPLOYEE;

/* ================================================
   REZULTAT:

   Prvi COUNT: 104 zaposlena
   Drugi COUNT: i dalje 104 zaposlena
   Iako je Sesija B ubacila NOVOG zaposlenog (EMP_NO = 999)
   i potvrdila ga COMMIT-om, Sesija A ga NE VIDI.

   Ovo je dokaz da SNAPSHOT izolacija:
   - "Zamrzava" stanje baze na trenutak pokretanja transakcije
   - Potpuno sprečava anomaliju FANTOMSKO ČITANJE
   ================================================= */
```

Primer 4: Update Conflict

Opis: Sprečavanje dve transakcije da istovremeno menjaju isti red

Rezultat: Firebird koristi pesimističko zaključavanje na nivou verzije zapisa kako bi sprečio gubitak izmena.

Izvorni kod: Update Conflict

```
/* ================================================
   Primer 5: DEADLOCK između dve transakcije

   Situacija:
   - Transakcija A zaključava red 2, pokušava da zaključa red 4
   - Transakcija B zaključava red 4, pokušava da zaključa red 2
   ------------------------------------------------- */
/* Prvo, Transakcija A ažurira red 2 */
UPDATE EMPLOYEE
SET PHONE_EXT = '111'
WHERE EMP_NO = 2;
/* -- NE RADIMO COMMIT -- */
/* Transakcija B ažurira red 4 */
UPDATE EMPLOYEE
SET PHONE_EXT = '222'
WHERE EMP_NO = 4;
/* -- NE RADIMO COMMIT -- */

/* Transakcija A pokušava da ažurira red 4 (ali red 4 je zaključan
   od strane B) */
UPDATE EMPLOYEE
SET PHONE_EXT = '111'
WHERE EMP_NO = 4;
```

```

/* Transakcija B pokušava da ažurira red 2 (ali red 2 je zaključan od strane A) */
UPDATE EMPLOYEE
SET PHONE_EXT = '222'
WHERE EMP_NO = 2;

/* ================================================
   REZULTAT:
   Firebird Lock Manager detektuje mrtvu petlju
   i prekida JEDNU od transakcija (obično onu koja je poslednja pokušala)

   Greška koju će jedna transakcija dobiti:
   "deadlock"
   – update conflicts with concurrent update
   ================================================ */
```

Primer 5: Verifikacija Lock Timeout mehanizma

Opis: Želimo da testiramo koliko dugo transakcija čeka na oslobađanje resursa

Rezultat: Ukoliko Sesija A ne oslobodi red u roku od 5 sekundi, Sesija B prekida izvršavanje uz uz poruku **"lock conflict"** ili **"deadlock"** (zavisno od verzije)

Izvorni kod: Lock Timeout mehanizam

```
-- sesija A
UPDATE EMPLOYEE SET PHONE_EXT = '111' WHERE EMP_NO = 2;
-- nemojte komitovati promenu
-- sesija B
SET TRANSACTION WAIT LOCK TIMEOUT 5;
UPDATE EMPLOYEE SET PHONE_EXT = '222' WHERE EMP_NO = 2;
-- sacekajte 5 sekundi
```

Primer 6: Pesimističko zaključavanje (SELECT ... WITH LOCK)

Opis: Želimo da vidimo kako funkcioniše pesimističko zaključavanje, odnosno SELECT … WITH LOCK. Sesija A treba da pročita red zaposlenog gde je EMP_NO = 2 sa korišćenjem WITH LOCK klauzule, dok Sesija B pokušava da pročita ovaj red isto stavljajući LOCK

Rezultat: Sesija A će staviti prva ekskluzivni katanac na taj red, pa će Sesija B pri pokušaju da uradi isto dobiti grešku o deadlock-u. Dakle, SELECT WITH LOCK implementira pesimističko zaključavanje — red se rezerviše odmah pri čitanju, što garantuje da niko ne može da ga promeni dok ga prva sesija ne oslobodi. Ovaj pristup je suprotan od podrazumevanog optimističkog modela Firebirda i koristi se samo kada je verovatnoća konflikta visoka (npr. prodaja poslednje karte).

Izvorni kod: Lock Timeout mehanizam

```
/* ================================================
   Primer 6: LOCK TIMEOUT mehanizam – čekanje sa vremenskim ograničenjem
   ================================================

   Sesija A (prvi prozor) – prva transakcija zaključava red
   ------------------------------------------------- */
UPDATE EMPLOYEE
SET PHONE_EXT = '111'
WHERE EMP_NO = 2;

/* NE RADIMO COMMIT! Sesija A drži zaključan red 5 sekundi
  Čekamo 5 sekundi da simulira sporu transakciju

   Sesija B (drugi prozor) – pokušava da menja ISTI red, ali sa timeout-om
   Podešavamo transakciju da čeka NAJVIŠE 3 sekunde */
SET TRANSACTION WAIT LOCK TIMEOUT 3;

UPDATE EMPLOYEE
SET PHONE_EXT = '222'
WHERE EMP_NO = 2;

/* ================================================
   REZULTAT:

   Sesija B će čekati 3 sekunde da se red oslobodi.
   Pošto Sesija A ne oslobađa red (traje 5 sekundi),
   posle 3 sekunde Sesija B dobija grešku: "lock time-out"
   – lock time-out on wait transaction
   UPDATE se ne izvršava!
   ================================================ */


/* ------------------------------------------------
   Varijanta 2: Isti primer, ali sa DOVOLJNO vremena
   Sesija A (prvi prozor) */
UPDATE EMPLOYEE
SET PHONE_EXT = '111'
WHERE EMP_NO = 2;
/* NE RADIMO COMMIT, ali ćemo uraditi u intervalu od 5 sekundi
   Sesija B (drugi prozor) – čeka 10 sekundi */
SET TRANSACTION WAIT LOCK TIMEOUT 10;

```

```
UPDATE EMPLOYEE
SET PHONE_EXT = '222'
WHERE EMP_NO = 2;

/* ================================================
   REZULTAT (ako Sesija A uradi COMMIT u roku):
   Sesija B čeka, Sesija A uradi COMMIT posle 2 sekunde,
   red se oslobađa, Sesija B nastavlja i USPEŠNO menja podatak.
   Konačna vrednost: '222'
   ================================================ */
```

9. Zaključak

Istraživanje sprovedeno u okviru ovog rada je potvrdilo da je Firebird, iako se često svrstava u kategoriju "lakših" relacionih baza podataka, zadržao jedan od najsofisticiranijih i najdoslednijih transakcionih modela u industriji. Specifičan put koji je ovaj sistem izabrao kroz implementaciju **Multi-Generational Architecture (MGA)** predstavlja inženjersko rešenje koje na elegantan način balansira između strogih teoretskih zahteva ACID principa i praktičnih potreba za visokim performansama u višekorisničkom radu.

Ključni zaključak analize unutrašnje strukture Firebirda je da njegova snaga leži u eliminaciji konflikata između čitača i pisača. U svetu gde se zahtevi za generisanjem izveštaja u realnom vremenu često preklapaju sa intenzivnim unosom podataka, Firebird-ov MVCC (Multi-Version Concurrency Control) model nudi prednost koju je teško nadmašiti tradicionalnim mehanizmima zaključavanja. Činjenica da dugačke `Snapshot` transakcije mogu da čitaju konzistentno stanje baze bez postavljanja ijednog katanca na operativne tabele, omogućava razvoj sistema koji su istovremeno robusni i responsivni.

Međutim, praktični testovi sprovedeni u radu jasno su ukazali na to da ova arhitektonska sloboda nosi sa sobom i specifične inženjerske odgovornosti. **Garbage Collection** mehanizam, iako automatizovan, nije svemoćan. On direktno zavisi od discipline klijentskih aplikacija i ispravnog upravljanja životnim ciklusom transakcija. Nagomilavanje starih verzija zapisa, uzrokovano "zaboravljenim" ili predugačkim transakcijama, identifikovano je kao najveći rizik po performanse sistema. Iz toga proističe kritičan zaključak: uspeh implementacije Firebird baze podataka ne zavisi samo od dizajna šeme i kvaliteta upita, već primarno od kontrole transakcionih markera poput `OAT` i `OIT`.

Takođe, evolucija sistema kroz verziju 4.0 i uvođenje **Read Committed Read Consistency** nivoa izolacije pokazuje da zajednica programera aktivno radi na rešavanju najveće mane optimističkog modela – učestalih konflikata pri upisu. Ovim dodatkom, Firebird se približio modernim „cloud-scale” zahtevima, gde se očekuje da baza sama rešava sitne sukobe na nivou instrukcije, smanjujući potrebu za kompleksnom logikom ponavljanja transakcija unutar samog aplikativnog koda.

Na kraju, može se konstatovati da Firebird transakcioni model zahteva "promenu paradigme" kod inženjera koji dolaze iz svetova gde dominira strogo zaključavanje. On zahteva proaktivno praćenje zdravlja baze kroz sistemske monitoring tabele i pažljivo biranje parametara unutar **Transaction Parameter Block-a**. Za master nivo projektovanja informacionih sistema, Firebird predstavlja izuzetan primer kako se sofisticirana arhitektura može iskoristiti za postizanje maksimalne konkurentnosti, pod uslovom da se razumeju i poštuju pravila po kojima funkcioniše njegov "mehanizam generacija". Budućnost ovog sistema, sudeći po trendovima u novijim verzijama, ide ka još većoj automatizaciji održavanja, čime će se dodatno smanjiti jaz između njegove unutrašnje kompleksnosti i lakoće korišćenja u produkcionim uslovima.

10. Literatura

[1] **ANSI/ISO/IEC 9075-2:2016**. Information technology – Database languages – SQL – Part 2: Foundation (SQL/Foundation). (Međunarodni standard za SQL nivoe izolacije).

[2] **Borrie, H. (2004).** The Firebird Book: A Reference for Database Developers. Apress. (Osnovna literatura za unutrašnju arhitekturu i MGA mehanizam).

[3] **Firebird Project.** Firebird 4.0 Language Reference. Zvanična dokumentacija dostupna na: firebirdsql.org.

[4] **Firebird Project (2021).** Firebird 4.0 Release Notes: Monitoring Enhancements and Read Consistency. (Tehnička dokumentacija za verziju 4.0+).

[5] **Firebird SQL.** Transaction Inventory Page (TIP) internals. Tehnički članci o unutrašnjoj strukturi stranica inventara transakcija.

[6] **Kuznetsov, D.** Understanding Firebird Garbage Collection. Tehnička dokumentacija o mehanizmima čišćenja i održavanja baze podataka.

[7] **Sistemske tabele (Monitoring RDB$ i MON$)** – Interna dokumentacija metapodataka Firebird engine-a.